

Reviewer Pack — Minimal Geometric Entropy in Symplectic Geometry and Communi...

Entry 3691637e99e3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 06:35:55 UTC

Minimal Geometric Entropy in Symplectic Geometry and Communication Complexity Rank

Entry ID: 3691637e99e3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 06:35:55 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity

Statement:

For any given boolean function f with n variables, the communication complexity rank of f 's associated decision tree is linearly correlated with the minimal geometric entropy of the symplectic leaves that arise from the symplectic reduction of the moment map corresponding to f 's Boolean cube, such that $\text{communication_complexity_rank}(f) = \Theta(\text{minimal_geometric_entropy}(n))$.

Rationale (proposer's reasoning):

Symplectic geometry provides a geometric framework for understanding the structure of complex systems. By mapping boolean functions to symplectic structures, we may uncover hidden connections between geometric invariants and computational complexity, potentially leading to new complexity-theoretic insights.

Taxonomy category: symplectic_geometry_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 44f5fbb816f9cc00

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated boolean functions with n variables ($n \leq 40$), the correlation coefficient between communication complexity rank and minimal geometric entropy across 30 seeds is greater than or equal to 0.7 and no seed shows a negative correlation. The conjecture is falsified if the correlation coefficient is less than 0.5 or any seed produces a negative correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "communication complexity rank" AND "symplectic geometry" AND minimal geometric entropy" - "moment map" AND symplectic reduction AND boolean cube communication complexity" - "decision tree" AND boolean function AND symplectic leaves minimal geometric entropy

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = int(math.log2(len(f)))
        # Simplified decision tree construction
        if n == 1:
            return abs(f[0] - f[1])
        else:
            left = communication_complexity_rank([f[i] for i in range(2**(n-1))])
            right = communication_complexity_rank([f[i + 2**(n-1)] for i in range(2**(n-1))])
            return max(left, right) + 1

    def minimal_geometric_entropy(n):
        # Simplified geometric entropy calculation
        return n * math.log2(n)

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = generate_boolean_function(n)
        rank = communication_complexity_rank(f)
        entropy = minimal_geometric_entropy(n)
```

```

        results.append((rank, entropy))

    if not results:
        return {
            "metric_name": "communication_complexity_rank",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "no_results"
        }

    ranks, entropies = zip(*results)
    correlation_coefficient = sum((ranks[i] - mean(ranks)) * (entropies[i] - mean(entropies)) for
                                i in range(len(ranks))) / (len(ranks) - 1)

    return {
        "metric_name": "communication_complexity_rank",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n for _, n in results),
        "conjecture_holds": 0.5 < correlation_coefficient <= 0.7 and all(correlation_coefficient >
                                0.5 for r in results),
        "counterexample": "" if 0.5 < correlation_coefficient <= 0.7 else f"correlation_coefficient={correlation_coefficient}"
    }

def mean(lst):
    return sum(lst) / len(lst)

def variance(lst):
    avg = mean(lst)
    return sum((x - avg) ** 2 for x in lst) / len(lst)

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = mean([r["metric_value"] for r in results if r["metric_value"] is not None])
    std_value = math.sqrt(variance([r["metric_value"] for r in results if r["metric_value"] is not None]))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and any(r["counterexample"] != "" for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_data n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14558
2	preregistration	ollama_remote	glm4:latest	0	0	9523
3	novelty	ollama_remote	glm4:latest	0	0	14508
4	novelty	ollama_remote	glm4:latest	0	0	27211
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17384
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	60666
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40604
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15969
9	judge	ollama_remote	glm4:latest	0	0	44409

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 244831 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/3691637e99e3.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/3691637e99e3.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/3691637e99e3.tar.gz* (if generated)